RiderDrop Mini-Sprint: Where Did the Cash Go?

45 minutes · solo or pairs · Python + SQL Server

The scenario

The client. RiderDrop (fictional) is a last-mile courier in Karachi. 25 riders deliver parcels across 4 zones. Most parcels are **COD** — cash on delivery — so the rider collects cash at the door and remits it to the office the same night.

The problem. September's cash book is short. Operations says every parcel was delivered. Finance says the cash never arrived. Both are reading different exports.

Your mission. Build a small pipeline that loads three messy exports, cleans them, and tells Finance exactly how much cash is missing and who is holding it.

Stack. Python 3 (pandas, SQLAlchemy + pyodbc) and SQL Server — Express, Developer or Azure SQL, all fine. T-SQL for every transformation that can be SQL.

Setup (do this before the clock starts)

```
CREATE DATABASE riderdrop;
GO
USE riderdrop;
GO
CREATE SCHEMA raw;    GO
CREATE SCHEMA clean;  GO
CREATE SCHEMA dq;     GO
```

```
from sqlalchemy import create_engine
engine = create_engine(
    "mssql+pyodbc://@localhost\\SQLEXPRESS/riderdrop"
    "?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes",
    fast_executemany=True,   # without this, loading 3,870 rows crawls
)
```

Driver 18 users: add `&TrustServerCertificate=yes`. SQL auth instead of Windows auth: `mssql+pyodbc://user:pass@host/riderdrop?driver=...`.

Check your setup before the clock starts:

```
pip install -r requirements.txt
Copy-Item .env.example .env
python make_data.py            # generates the three CSVs
python 00_test_connection.py   # every line must say OK
```

`.env` holds your server and database name. Leave `SQL_USER` blank to use **Windows authentication** — your own login, nothing to type. If your class shares one server, set `SQL_DATABASE=riderdrop_yourname` or you will all overwrite each other.

The data

Run `python make_data.py` to generate `data/` (3 files, ~5,800 rows total).

File	Rows	What's messy
`parcels.csv`	~1,240	40 duplicate `parcel_id` (keep latest `updated_at`), three date formats, zone names in mixed case with stray spaces
`scans.csv`	~3,870	Duplicate scans seconds apart, scans for parcel IDs that don't exist, parcels with no final scan
`cod_remittances.csv`	~734	Amounts as text (`Rs 1,250`, `1250.00`, `" 1250 "`), remittances for parcels that were never delivered

Business rules

1. If a `parcel_id` appears twice in `parcels.csv`, keep the row with the latest `updated_at`.
2. Two scans for the same parcel with the same `status` within **2 minutes** count as one.
3. A parcel's **final status** is the status of its latest scan.
4. `cod_due` = the parcel's `cod_amount` **only if** final status is `delivered`. Otherwise 0.
5. **On time** = delivered on or before `promised_date`.
6. **Shortfall** = `cod_due` − total remitted for that parcel. Negative shortfall (over-remitted) is still a finding.

Stages

Stage 1 — Load (10 min)

- Load all three CSVs into `raw.parcels`, `raw.scans`, `raw.remittances`, **every column as `NVARCHAR(200)`**. Read the CSVs with `dtype=str` so pandas doesn't guess types for you.
- Re-running the script must not duplicate rows (`if_exists="replace"` is the lazy way; `TRUNCATE TABLE` then insert is the honest one).

- Print row count per file and per table; they must match.

Stage 2 — Clean (15 min)

Build `clean.parcels`, `clean.scans`, `clean.remittances` with real types (`DATE`, `DATETIME2`, `DECIMAL(10,2)`). - Dedupe parcels by rule 1 — `ROW_NUMBER() OVER (PARTITION BY parcel_id ORDER BY updated_at DESC)`. Keep a `clean.merge_log` table saying which `parcel_id`s were dropped. - Parse all three date formats to a real `DATE`. - Normalise zone names (trim, title case). - Drop duplicate scans by rule 2 — `LAG()` over `(parcel_id, status) ORDER BY scanned_at`. - Scans whose `parcel_id` is not in `clean.parcels` go to `clean.rejected_scans` with a reason. Do not silently drop them. - Parse the money column to `DECIMAL(10,2)`. - Build `clean.parcel_final` — one row per parcel: final status, delivered date, rider, zone, `cod_due`. Put a primary key on `parcel_id` and let SQL Server prove it's unique.

Stage 3 — Answer (15 min)

One `.sql` file per question, results saved as CSV.

1. Parcel count by zone and final status.
2. On-time delivery rate by zone.
3. Top 5 riders by total COD shortfall.
4. Parcels with a remittance but a final status that is **not** `delivered`.
5. Delivered COD parcels with **no** remittance — count and total rupees at risk.

Data quality (5 min)

Write three checks into a `dq.results` table (`check_name`, `passed`, `detail`, `run_at`): - `parcel_id` is unique and not null in `clean.parcels`. - `raw.scans` count = `clean.scans` + duplicates removed + `clean.rejected_scans`. - No parcel has `delivered_at` before its `booked_date`.

Deliverables

```
riderdrop-sprint/
├── README.md        # how to run, connection string, one line per cleaning decision
├── setup.sql        # database + schemas
├── pipeline.py      # Stages 1-2
├── queries/         # q1..q5.sql + results/*.csv
└── findings.md      # 3 bullets: the number, the query, the action
```

findings.md is the whole point. Three bullets, each one sentence with a rupee figure and what Finance should do on Monday.

Rules

- Do not edit the CSVs by hand. Every fix lives in code.
- No `SELECT *` into Excel for the answers. Every number comes from a `.sql` file in the repo.
- AI assistants allowed — but you must be able to explain any line you wrote.
- Ask about business rules, not about solutions.

Scoring (50 points)

Criterion	Points
Correct answers to Q1–Q5	20
Cleaning: every planted issue found and fixed in code	15
Data quality checks run and reconcile	5
Findings: three numbers with actions	10

Hints

- Profile first: `df.head()`, `value_counts()`, `isna().sum()` on all three files. Five minutes here saves twenty later.
- Dates — **the trap in this dataset.** `COALESCE` three explicit `TRY_CONVERT` styles: `23` for `2026-09-21`, `103` for `20/09/2026`, `106` for `20-Sep-2026`. The one that matches wins; the others return `NULL` instead of blowing up. `sql COALESCE(TRY_CONVERT(DATE, booked_date, 23), TRY_CONVERT(DATE, booked_date, 103), TRY_CONVERT(DATE, booked_date, 106))` Do **not** reach for `pd.to_datetime(col, format='mixed', dayfirst=True)`. It looks like the tidy one-liner and it is wrong here: it reads `11/09/2026` as 9 November, not 11 September. Nothing warns you — you just get parcels delivered two months before they were booked. If you clean in pandas, pass each format explicitly and keep whichever parses. Either way, add a check that no `delivered_date` precedes its `booked_date`; that is the only reason anyone notices.
- Money: `TRY_CONVERT(DECIMAL(10,2), REPLACE(REPLACE(amount, 'Rs', ''), ',', ''))`.
- Dedupe scans (T-SQL — `DATEDIFF`, not `julianday`): `sql WITH gapped AS ( SELECT *, LAG(scanned_at) OVER (PARTITION BY parcel_id, status ORDER BY scanned_at) AS prev_scan FROM clean.scans ) SELECT * FROM gapped WHERE prev_scan IS NULL OR DATEDIFF(SECOND, prev_scan, scanned_at) > 120;`
- Final status per parcel: `ROW_NUMBER() OVER (PARTITION BY parcel_id ORDER BY scanned_at DESC) = 1`.
- Export a result to CSV from Python: `pd.read_sql(open('queries/q3.sql').read(), engine).to_csv(...)`.
- The story is in Q3, Q4 and Q5. Start your findings there.